

Term Project: SLR Parser Construction

1. Project Overview

In this term project, you will implement a bottom-up SLR(1) parser for an ambiguous toy language. This exercise consolidates your understanding of parsing theory and compiler implementation by guiding you through:

- Grammar analysis and ambiguity detection
- Grammar refactoring to eliminate ambiguity
- Construction of SLR(1) parsing table
- Parser implementation including parse-tree generation and error reporting

2. Project Requirements

- **Grammar Refactoring**
 - Below, an **ambiguous CFG** is provided (see **3. Grammar Specification**).
 - Perform disambiguation and document the process; submit your refactored grammar and the process in your report.
- **SLR Table Construction**
 - Compute and present FIRST and FOLLOW sets for all nonterminals of the disambiguated grammar in your report.
 - Build the SLR(1) parsing table (ACTION and GOTO table).
 - In the report, provide an example of how FOLLOW sets resolve any shift/reduce or reduce/reduce conflicts.
 - You can use this website: <https://jsmachines.sourceforge.net/machines/slr.html>
- **Parser Implementation**
 - Implement **parser** function in Python language (see **4. Code Instructions**).
 - On acceptance, produce a parse tree; on rejection, generate a detailed error report indicating token index and cause.

3. Grammar Specification

The following CFG defines a C-like language with declarations, control flow, expressions, and function calls. It is intentionally ambiguous.

1.	Program	→ DeclList
2.	DeclList	→ Decl DeclList ϵ
3.	Decl	→ VarDecl FuncDecl
4.	VarDecl	→ type id ;
5.	VarDecl	→ type id = Expr ;
6.	FuncDecl	→ type id (ParamList) Block
7.	ParamList	→ Param , ParamList Param ϵ
8.	Param	→ type id
9.	Block	→ { StmtList }
10.	StmtList	→ Stmt StmtList ϵ
11.	Stmt	→ if (Expr) Stmt
12.	Stmt	→ if (Expr) Stmt else Stmt
13.	Stmt	→ while (Expr) Stmt
14.	Stmt	→ for (Expr ; Expr ; Expr) Stmt
15.	Stmt	→ return Expr ;
16.	Stmt	→ VarDecl
17.	Stmt	→ ExprStmt
18.	Stmt	→ Block
19.	ExprStmt	→ id = Expr ;
20.	Expr	→ Expr == Expr
21.	Expr	→ Expr + Expr
22.	Expr	→ Expr * Expr
23.	Expr	→ - Expr
24.	Expr	→ id (ArgList)
25.	Expr	→ id
26.	Expr	→ num
27.	Expr	→ (Expr)
28.	ArgList	→ Expr , ArgList Expr ϵ

All tokens starting with a capital letter (e.g., Program, DeclList, Expr) are non-terminals. All other tokens (e.g., id, num, or, ==, (, ;, etc.) are terminals. Whitespace between symbols in each production rule is used for formatting.

4. Code Instructions

You are required to use the provided **Python code structure** to run your parser.

You will be provided with two Python files:

- `main.py`: the main entry point of the program (DO NOT modify this file)
- `parser.py`: the file where you will implement the `parser()` function and any supporting classes or functions

Your main entry point will be `main.py`. It reads the token input file, calls your `parser()` function in `parser.py`, and prints the result.

Important: You are allowed to only use Python's standard library. DO NOT use any external packages.

5. Team Formation

It is allowed to build a team of up to two students. If you want to form your team, submit your team information by **May 16 (Fri) 23:59** using the provided Google Form link (<https://forms.gle/d2tqbjtNzFbzKdWY6>).

You may choose one of the following options:

- Work individually
- Form a team of two with a partner of your choice
: In this case, only one member needs to fill out the Google Form.
- Join a randomly matched team of two
: If the number of students is odd, some students may be selected to work individually by lottery.

If you do not submit your team preference by May 16, you will be automatically assigned to **work individually** without exception.

The finalized team list and team ID assignments will be announced on May 19 (Mon).

6. Submission Deadline & Deliverables

Deadline: May 31 (Sat) 23:59 (through our E-class system)

For a delayed submission, you will lose $0.1 \times$ score per day.

Deliverable 1.

A compressed code file named `team_<teamID>.zip` containing:

(Please DO NOT change the below deliverables' file names)

- `main.py` (provided entry-point file, unmodified)
- `parser.py` (your implementation)

Deliverable 2.

`team_<teamID>_report.pdf`, which must include:

- A clear description of grammar disambiguation process and your full disambiguated grammar
- FIRST and FOLLOW sets for all nonterminals
- SLR(1) parsing table (ACTION and GOTO)
- At least one example where FOLLOW sets resolve a conflict in the table
- Several example of code results: your own test token streams with their corresponding parse tree or error report
- The report can be written in either English or Korean.

If you find any error in the grammar or project description,
please contact the instructor via email: hsmoon@cau.ac.kr

Plagiarism in either the report or code will result in an **automatic F grade**, regardless of the reason. You may consult tools like ChatGPT for guidance, but do not copy or submit generated content as-is. This may lead to high similarity across submissions, which will be treated as a plagiarism offense.